Semantic tagging

AI Chat Page — Integration Design · built Nov 13, 2026

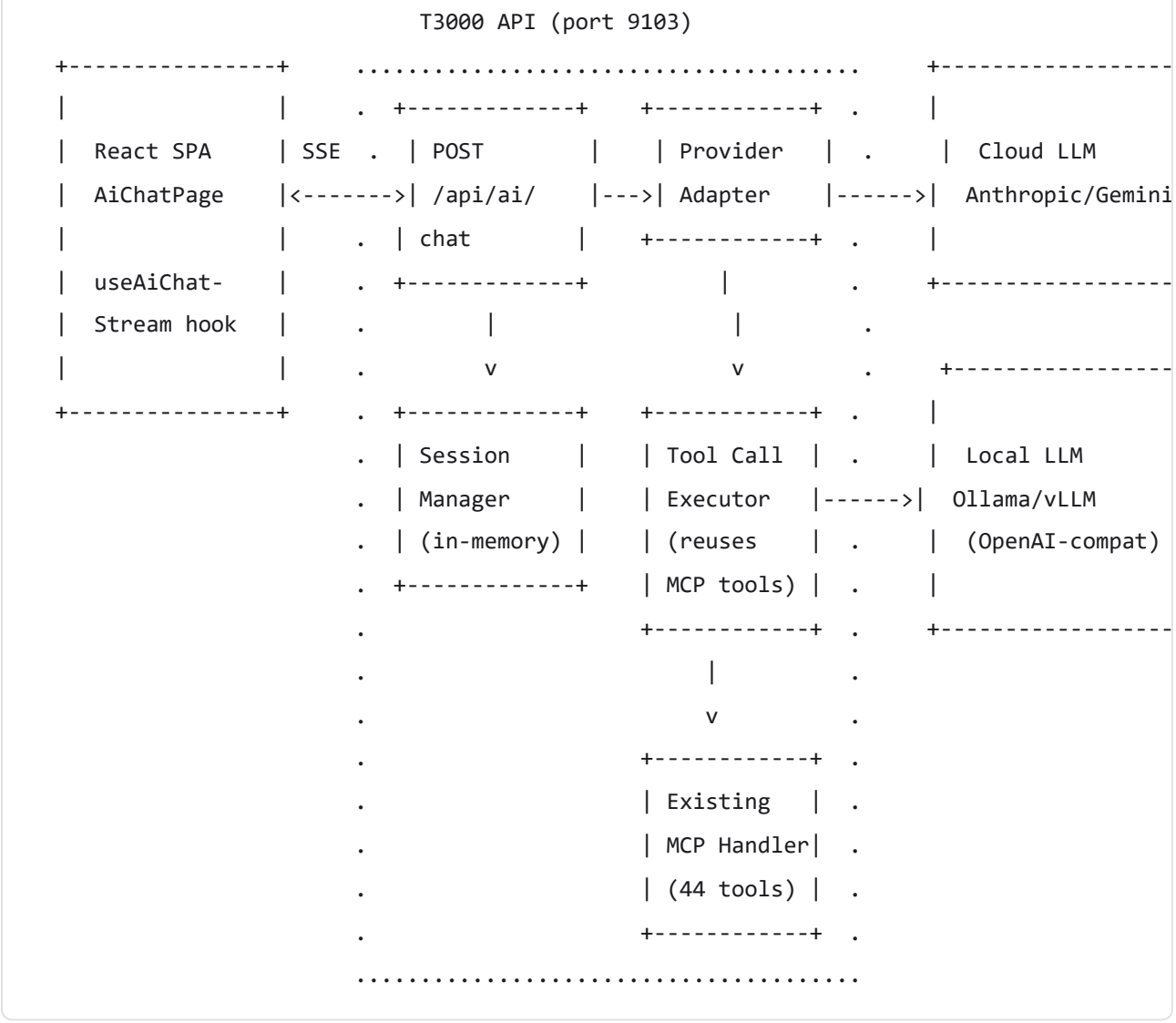
AI Chat Page — Integration Design

> Back to AI Docs | Local LLM Setup | Cloud LLM APIs

Complete integration plan for the built-in AI chat page. Covers backend SSE endpoint, LLM provider abstraction, tool-call loop, and React frontend.

1. Overall Architecture

TEXT



Key principle: The browser talks only to T3000 via SSE. T3000 proxies to the LLM, intercepts tool calls, executes them locally against the existing MCP machinery, and streams text back. No API keys ever reach the browser.

---

2. Backend Design

2.1 New Module: `api/src/ai/`

```
TEXT

api/src/ai/
  mod.rs              -- module root, re-exports create_ai_routes()
  types.rs            -- ChatRequest, Message, StreamEvent, AiSettings
  session.rs          -- in-memory session store (Arc<Mutex<HashMap>>)
  tool_executor.rs    -- maps tool names -> existing MCP handler functions
  routes.rs           -- axum route handlers (SSE streaming)
  providers/
    mod.rs            -- LlmProvider trait + dispatch
    local.rs          -- OpenAI-compatible (Ollama, vLLM, LM Studio, llama.cpp)
```

2.2 SSE Protocol

`POST /api/ai/chat` returns `text/event-stream`. Events sent from server to client:

```
TEXT

Event              Direction     Meaning
_____________________________________________________________________

text_delta         S -> C        Streaming text chunk, append to UI
tool_call          S -> C        LLM requested a tool, show pending badge
tool_result        S -> C        Tool executed locally, show summary
done               S -> C        Turn complete, contains session_id
error              S -> C        Fatal error, stop streaming
```

Request body (POST /api/ai/chat):

JSON

```
{
  "provider": "local",
  "model": "llama3.1:8b",
  "messages": [
    { "role": "user", "content": "What is AHU-1 supply temperature?" }
  ],
  "session_id": "optional-uuid-here",
  "settings": {
    "endpoint": "http://localhost:11434/v1",
    "api_key": ""
  }
}
```

SSE stream example (raw):

TEXT

```
data: {"event":"text_delta","content":"Let me"}

data: {"event":"text_delta","content":" check the"}

data: {"event":"tool_call","id":"call_01","name":"t3000_point_read","args":"{\"serial_nu

data: {"event":"tool_result","id":"call_01","result":"[{\"label\":\"AHU-1 Supply Temp\",

data: {"event":"text_delta","content":"The supply air"}

data: {"event":"text_delta","content":" temperature is 55.2"}

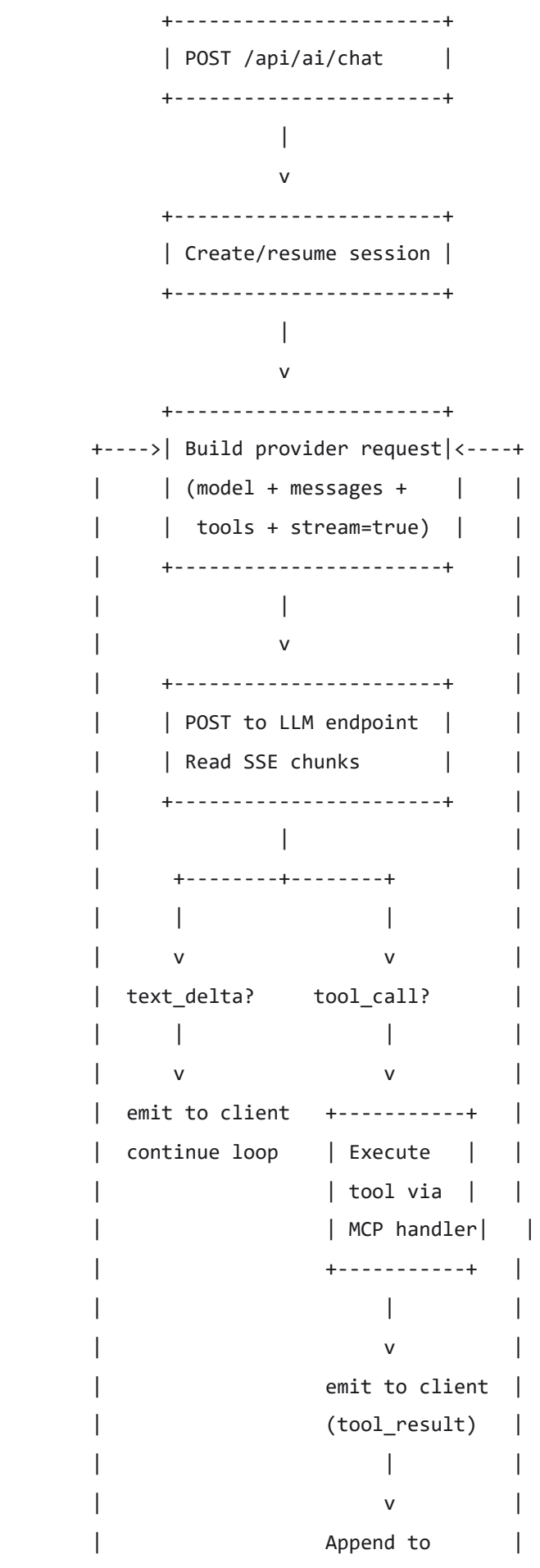
data: {"event":"text_delta","content":" Deg.C."}

data: {"event":"done","session_id":"abc123-def456"}
```

2.3 Tool-Call Loop (Route Handler)

The route handler manages the loop — NOT the provider. This keeps providers stateless and simple.

TEXT


POST /api/ai/chat
Create/resume session
Build provider request
(model + messages +
tools + stream=true)
POST to LLM endpoint
Read SSE chunks
text_delta?
tool_call?
emit to client
continue loop
Execute
tool via
MCP handler
emit to client
(tool_result)
Append to



2.4 Tool Executor Strategy

Two options — **Option A is recommended:**

Option	Approach	Pros	Cons
A: Direct	Call existing MCP handler functions directly	Zero overhead, typesafe	Requires `pub(crate)` refactor of `mcp.rs` dispatch
B: JSON-RPC	Construct synthetic `JsonRpcRequest`, call dispatch	No refactor needed	Double-serialization, error wrapping noise

Recommendation: Option A. Extract each `"tool_name" => { ... }` match arm in `mcp.rs` into a `pub(crate)` function. The tool executor calls these directly. This is a mechanical refactor — no logic changes.

2.5 Session Manager

TEXT

```
+-----------------------------------------------------+
| SessionManager (Arc<Mutex<HashMap<String,Session>>>)|
+-----------------------------------------------------+
| create(req: &ChatRequest) -> Session                |
| get(id: &str) -> Option<Session>                    |
| update_messages(id, messages)                       |
| delete(id)                                          |
| cleanup_expired(max_age: Duration)                   |
+-----------------------------------------------------+

Session {
    id: String,
    provider: String,
    model: String,
    endpoint: String,
    api_key: Option<String>,    // Never logged
    messages: Vec<Message>,     // Full history for tool-call loop
    created_at: Instant,
    last_active: Instant,
}
```

- Max 100 concurrent sessions
- Idle timeout: 1 hour (auto-purge via background tokio task every 5 min)
- Sessions survive server restart only if saved to DB (Phase 4 polish — not Phase 1)

2.6 Settings Storage

TEXT

```
+-------------------+       +------------------+       +------------------+
| AI Chat Page (UI) | <---- | GET /api/ai/     | <--- | settings table   |
|                   |       | settings         |       | (encrypted       |
|                   | ----> | PUT /api/ai/     | ---> |  api_key column) |
|                   |       | settings         |       +------------------+
+-------------------+       +------------------+

API key flow:
  1. User enters key in Settings panel
  2. Frontend POSTs to PUT /api/ai/settings
  3. Backend encrypts key with AES-256-GCM before storing
  4. Frontend receives { ok: true } (key masked to "sk-ant...****")
  5. On subsequent requests, frontend sends NO key — backend injects it
```

2.7 Provider Adapter Trait

RUST

```rust
// providers/mod.rs

#[async_trait]
pub trait LlmProvider: Send + Sync {
    /// Stream chat completion. The provider ONLY handles the HTTP request to
    /// the LLM and parses the SSE stream. Tool-call looping is done by the
    /// route handler — the provider just emits tool_call events.
    async fn stream_chat(
        &self,
        endpoint: &str,
        api_key: Option<&str>,
        model: &str,
        messages: &[Message],
        tools: &[ToolDef],          // Canonical tool definitions
        tx: Sender<StreamEvent>,    // Send events back to route handler
    ) -> Result<(), AiError>;
}
```

Three implementations:

- `LocalProvider` — OpenAI-compatible `/v1/chat/completions`

- `AnthropicProvider` — `/v1/messages` (Phase 3)
- `GeminiProvider` — `streamGenerateContent` (Phase 3)

2.8 Routes to Wire

In `server.rs` (after the MCP routes line):

```rust
.merge(crate::ai::create_ai_routes())
```

In `lib.rs`:

```rust
pub mod ai;   // Add after `pub mod haystack;`
```

2.9 Additional API Endpoints

```text
POST   /api/ai/chat          SSE streaming chat
DELETE /api/ai/sessions/:id  Clear a session
GET    /api/ai/settings      Get current provider/model/endpoint (key masked)
PUT    /api/ai/settings      Save provider settings (key encrypted)
GET    /api/ai/tools         List available tools (for debug/transparency)
```

---

3. Frontend Design

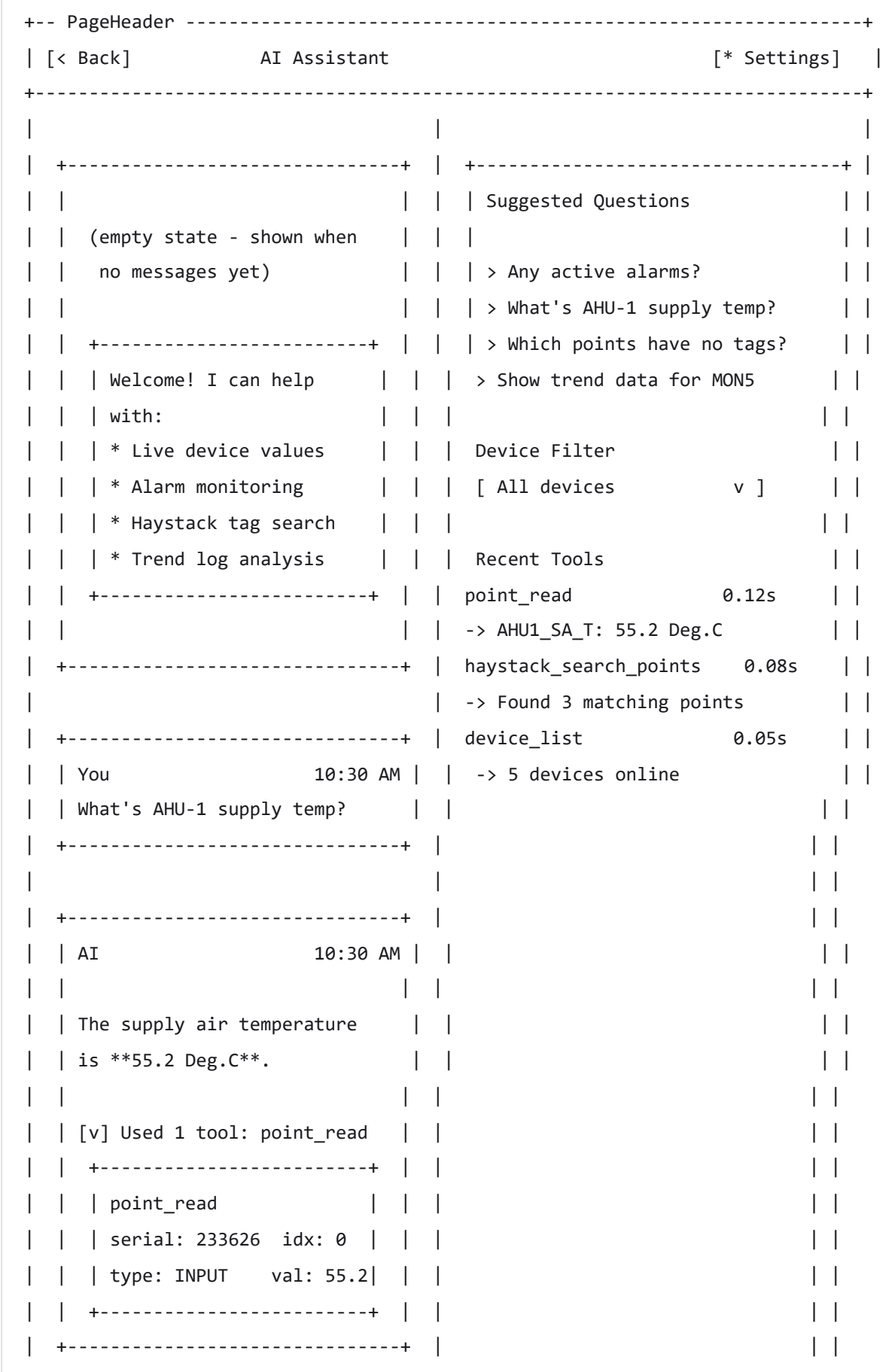
3.1 File Tree

TEXT

```
src/t3-react/features/ai-chat/
  index.ts                          -- barrel export
  pages/
    AiChatPage.tsx                   -- page component, layout
  hooks/
    useAiChatStream.ts               -- SSE client, message state machine
    useAiSettings.ts                 -- provider settings CRUD
  components/
    ChatPanel.tsx                    -- main chat area orchestrator
    ChatMessage.tsx                  -- single message bubble
    ChatInput.tsx                    -- input bar with send/stop
    SettingsDrawer.tsx               -- slide-over settings panel
    ContextPanel.tsx                 -- right sidebar (suggestions, device filter, tool
    EmptyState.tsx                   -- welcome screen with suggested questions
    ToolCallCard.tsx                 -- expandable tool call result inline
```

3.2 Page Layout (ASCII)

TEXT



```
|                                    |                                    | |
|  ... streaming indicator ...       |                                   | |
|  The supply fan is currently ▉     |                                   | |
|                                    |                                    | |
|  +-------------------------------+ |                                    | |
|  | Type a message...       [STOP] | |                                     | |
|  | local:llama3.1:8b  [v]         | |                                     | |
|  +-------------------------------+ |                                    | |
+------------------------------------+------------------------------------+
|  Collapse sidebar >                                                   <<  |
+-------------------------------------------------------------------------+
```

3.3 Component State Flow

TEXT

```
+----------+    +------------+    +------------------+    +---------+
| ChatInput |--->| ChatPanel  |--->| useAiChatStream  |--->| POST    |
| (user     |    | (orchestra-|    | (SSE reader,     |    | /api/ai/|
|  types    |    |  tor)      |    |  message store)  |    | chat    |
|  message) |    +------------+    +------------------+    +---------+
+----------+         |                    |
      ^              |                    |
      |              v                    v
      |       +------------+        +------------+
      +-------| ChatMessage|        | ToolCall   |
   (scroll    | (bubble    |        | Card       |
    back to   |  render)   |        | (expandable|
    input)    +------------+        +------------+
```

Message type in state:

TYPESCRIPT

```typescript
type ChatMessage =
  | { role: 'user';    content: string; timestamp: number }
  | { role: 'assistant'; content: string; toolCalls: ToolCallRecord[]; timestamp: number
  | { role: 'system';  content: string; timestamp: number };  // errors, info

type ToolCallRecord = {
  id: string;
  name: string;
  args: string;        // JSON
  result?: string;     // JSON, set when tool_result arrives
  status: 'pending' | 'success' | 'error';
};
```

3.4 SSE Client Hook (`useAiChatStream`)

TEXT

```
+-----------------------+
| useAiChatStream()     |
+-----------------------+
| messages: ChatMessage[]
| isStreaming: boolean
| streamingText: string         // partial text while streaming
| activeToolCalls: Map<id,Record>
| sessionId: string | null
+-----------------------+
| sendMessage(content): void
| abort(): void
| clearSession(): void
+-----------------------+

Internals:
  1. fetch('/api/ai/chat', { method:'POST', body: JSON })
  2. Get ReadableStream from response.body
  3. Pipe through TextDecoderStream
  4. Split on '\n\n', parse 'data: {...}' lines
  5. Dispatch by event type:
     text_delta  -> append to streamingText, update last assistant msg
     tool_call   -> insert ToolCallRecord with status:'pending'
     tool_result -> resolve ToolCallRecord, update status
     done        -> finalize message, store sessionId
     error       -> append system message with error text
  6. On stream end or abort: set isStreaming=false
```

3.5 Route & Toolbar Integration

Route: `/#/t3000/ai-chat` (under MainLayout, authenticated)

In `App.tsx` — add lazy import:

TYPESCRIPT

```typescript
const AiChatPage = React.lazy(() =>
  import('../features/ai-chat').then((m) => ({ default: m.AiChatPage }))
);
```

In `App.tsx` — add Route inside `<Route path="/t3000">`:

TYPESCRIPT

```tsx
<Route
  path="ai-chat"
  element={
    <React.Suspense fallback={<div>Loading...</div>}>
      <AiChatPage />
    </React.Suspense>
  }
/>
```

In `toolbarConfig.ts` — add button entry (before Array):

TYPESCRIPT

```ts
{
  id: 'toolbar-ai-chat',
  icon: ChatBubblesQuestionRegular,              // from @fluentui/react-icons
  label: 'AI Chat',
  tooltip: 'AI Assistant',
  action: 'openWindow',
  windowId: 16,
  route: '/t3000/ai-chat',
},
```

---

4. Complete Implementation Steps

Phase 1 — Core Plumbing (Local LLM, no tools)

TEXT

Step	File	Action
1.	api/src/ai/types.rs	CREATE — ChatRequest, Message, StreamEvent, AiSettings structs
2.	api/src/ai/session.rs	CREATE — SessionManager with create/get/delete/cleanup
3.	api/src/ai/providers/mod.rs	CREATE — LlmProvider trait
4.	api/src/ai/providers/local.rs	CREATE — LocalProvider (OpenAI compatible, SSE parsing)
5.	api/src/ai/routes.rs	CREATE — POST /api/ai/chat SSE handler, settings CRUD
6.	api/src/ai/mod.rs	CREATE — module root
7.	api/src/lib.rs	MODIFY — add `pub mod ai;`
8.	api/src/server.rs	MODIFY — merge ai routes
9.	src/.../ai-chat/hooks/useAiChatStream.ts	CREATE — SSE client hook
10.	src/.../ai-chat/hooks/useAiSettings.ts	CREATE — settings hook
11.	src/.../ai-chat/components/ChatMessage.tsx	CREATE — message bubble
12.	src/.../ai-chat/components/ChatInput.tsx	CREATE — input bar
13.	src/.../ai-chat/components/ChatPanel.tsx	CREATE — orchestrator
14.	src/.../ai-chat/components/EmptyState.tsx	CREATE — welcome screen
15.	src/.../ai-chat/pages/AiChatPage.tsx	CREATE — page layout
16.	src/.../ai-chat/index.ts	CREATE — barrel export
17.	src/t3-react/app/App.tsx	MODIFY — add lazy import + route
18.	src/t3-react/app/config/toolbarConfig.ts	MODIFY — add button

Phase 2 — Tool Calling

```
19.  api/src/ai/tool_executor.rs            CREATE — dispatch to MCP handlers
20.  api/src/haystack/mcp.rs                REFACTOR — extract handle_* fns
                                            as pub(crate) so tool_executor
                                            can call them directly
21.  src/.../ai-chat/components/ToolCallCard.tsx  CREATE — expandable card
22.  src/.../ai-chat/components/ContextPanel.tsx  CREATE — right sidebar
23.  api/src/ai/routes.rs                   MODIFY — add tool-call loop to
                                            SSE handler (the while loop)
```

Phase 3 — Cloud Providers + Settings

```
24.  api/src/ai/providers/anthropic.rs      CREATE — Anthropic adapter
25.  api/src/ai/providers/gemini.rs         CREATE — Gemini adapter
26.  src/.../ai-chat/components/SettingsDrawer.tsx CREATE — settings panel
```

Phase 4 — Polish

```
27.  Session persistence (save to DB, survive restart)
28.  Prompt template customization
29.  Suggested questions in empty state
30.  Device scoping (filter tools to one device)
31.  Mobile-responsive layout
```

---

5. Design Decisions & Rationale

5.1 Why SSE instead of WebSocket?

Feature	SSE	WebSocket
Direction	Server -> Client only (perfect for streaming)	Bidirectional
Reconnection	Built-in (EventSource API)	Manual
HTTP/2 multiplexing	Yes	No
Firewall friendly	Yes (plain HTTP)	Sometimes blocked
Complexity	Trivial	Moderate

SSE is the standard for LLM streaming. The frontend sends requests via `fetch POST` and receives responses via SSE. No WebSocket overhead needed.

5.2 Why in-memory sessions?

- Chat sessions are ephemeral — users don't expect history to survive a server restart
- Avoids database schema changes and migration complexity
- Simpler cleanup (just drop the map on restart)
- Max 100 sessions × ~50KB each = ~5MB memory — negligible
- Phase 4 can add DB persistence if needed

5.3 Why direct MCP handler calls (not JSON-RPC)?

The tool executor could construct a `JsonRpcRequest { method: "t3000_point_read", ... }` and call the existing MCP dispatch. But that means:

- Serialize args to JSON -> parse JSON -> dispatch -> serialize result -> parse result
- Two unnecessary (de)serialization passes

Instead, extract `pub(crate) async fn handle_point_read(args: Value, state: &T3AppState) -> Result<Value>` functions and call them directly. The MCP JSON-RPC endpoint still works — it just delegates to these same functions.

5.4 Why the route handler owns the tool-call loop?

Three options for tool-call loop placement:

Location	Pros	Cons
Provider	Self-contained	Provider must know about session, tool executor — violates SRP
Route handler	Clean separation, provider is stateless	Route handler is more complex
New "orchestrator" struct	Dedicated component	Over-engineering for v1

Route handler is the pragmatic choice. Providers become simple adapters that take messages + tools, return a stream of events. The route handler manages the session state and decides when to loop.

5.5 System Prompt Strategy

The system prompt is the most important factor for tool-calling quality. Default template (stored in settings, user-overridable):

```
TEXT
```

```
You are a building automation assistant for the T3000 platform.
You have access to tools that query real-time device data, search
Haystack semantic tags, read/write points, and check alarms.

Rules:
- ALWAYS use tools to fetch live data — never guess values.
- When asked about a device, use device_list first to find it.
- When asked about a point value, search by tags with
  haystack_search_points, then read with point_read.
- Keep responses concise. Include units when reporting values.
- If a tool fails, explain the error and suggest alternatives.
- For multi-point queries, use batch tools (point_read_batch).
```

6. Error Handling Matrix

TEXT

```
+------------------------------+----------------------------------------------+
| Scenario                     | User Experience                              |
+------------------------------+----------------------------------------------+
| LLM unreachable              | Red toast: "Cannot reach LLM server at       |
| (connection refused)         | localhost:11434. Is Ollama running?"          |
|                              | Input remains enabled for retry.             |
+------------------------------+----------------------------------------------+
| LLM returns non-200          | Toast with status + message. Input enabled.  |
+------------------------------+----------------------------------------------+
| Tool execution fails         | Tool result sent as error to LLM. LLM handles|
| (e.g., device offline)       | gracefully: "AHU-1 appears offline."         |
+------------------------------+----------------------------------------------+
| Session expired (>1h idle)   | Auto-creates new session. Subtle indicator:  |
|                              | "New session" badge appears briefly.         |
+------------------------------+----------------------------------------------+
| Network drops mid-stream     | "Connection lost" message. Retry button.     |
+------------------------------+----------------------------------------------+
| API key missing (cloud)      | Settings drawer auto-opens with focus on key |
|                              | field. Toast: "API key required for Claude." |
+------------------------------+----------------------------------------------+
| User sends while streaming   | Input disabled. Send button replaced with    |
|                              | [STOP] button.                               |
+------------------------------+----------------------------------------------+
| Rate limit hit (cloud)       | Error toast: "Rate limited. Retry in 30s."   |
|                              | Countdown shown.                             |
+------------------------------+----------------------------------------------+
| Very long tool result        | Truncated to 4KB in card. "Show full" button |
|                              | to expand. Always sent fully to LLM.         |
+------------------------------+----------------------------------------------+
```

7. Files Summary

TEXT

```
NEW FILES (14)
══════════════

Backend (7):
  api/src/ai/mod.rs
  api/src/ai/types.rs
  api/src/ai/session.rs
  api/src/ai/tool_executor.rs
  api/src/ai/routes.rs
  api/src/ai/providers/mod.rs
  api/src/ai/providers/local.rs

Frontend (7):
  src/t3-react/features/ai-chat/index.ts
  src/t3-react/features/ai-chat/pages/AiChatPage.tsx
  src/t3-react/features/ai-chat/hooks/useAiChatStream.ts
  src/t3-react/features/ai-chat/hooks/useAiSettings.ts
  src/t3-react/features/ai-chat/components/ChatPanel.tsx
  src/t3-react/features/ai-chat/components/ChatMessage.tsx
  src/t3-react/features/ai-chat/components/ChatInput.tsx

MODIFIED FILES (4)
══════════════════

  api/src/lib.rs                          -- add `pub mod ai;`
  api/src/server.rs                       -- merge ai routes
  src/t3-react/app/App.tsx                -- add lazy import + route
  src/t3-react/app/config/toolbarConfig.ts  -- add toolbar button

PHASE 1 SCOPE: 14 new + 4 modified = 18 files
```

---

8. Final Suggestions

8.1 Start with Phase 1 ONLY — get streaming working end-to-end first

The hardest part is the SSE plumbing (backend SSE handler + frontend ReadableStream parser). Get a simple "echo" working: user sends text, backend proxies to Ollama, streams text back. No tools, no settings panel, no sessions. Just:

1. Hardcoded endpoint `http://localhost:11434/v1`

2. Hardcoded model `llama3.1:8b`
3. Simple chat: POST -> stream -> display

Once that works, everything else builds on top.

8.2 Refactor mcp.rs BEFORE building tool_executor

The MCP dispatch in `mcp.rs` is a giant `match method_name { ... }` block. Before Phase 2, extract each arm into a named `pub(crate)` function. This is a pure refactor — no behavior change. It makes `tool_executor.rs` trivial.

8.3 Use a fixed system prompt initially

Don't make the system prompt configurable in Phase 1. Hardcode the building automation assistant prompt. Add the settings UI for it in Phase 4.

8.4 Test with a simple curl first

Before building any frontend, verify the backend works:

```shell
curl -N -X POST http://localhost:9103/api/ai/chat \
  -H "Content-Type: application/json" \
  -d '{"provider":"local","model":"llama3.1:8b","messages":[{"role":"user","content":"He
```

You should see SSE chunks streaming back. If this doesn't work, the frontend definitely won't.

8.5 Tool names exposed to LLM should match MCP exactly

The `name` field in tool definitions sent to the LLM must match the dispatch key in `tool_executor.rs`. Keep a single source of truth — ideally the `TOOLS` lazy_static in `mcp.rs`. Reference it when building the provider request.

8.6 Keep the right sidebar truly optional

The ContextPanel (suggestions, device filter, tool log) is useful but not essential for Phase 1. Defer it to Phase 2+. Phase 1 should be a clean two-element layout: messages + input.

8.7 Suggested test flow for Phase 1

TEXT

```
Install:  ollama pull llama3.1:8b
Run:      ollama serve
Verify:   curl http://localhost:11434/api/tags
Build:    cd api && cargo build
Test:     curl -N ... (see 8.4 above)
Build FE: yarn build   (or dev server)
Navigate: http://localhost:9103/#/t3000/ai-chat
Type:     "Hello!"
Expect:   Streaming AI response in chat bubbles
```

> **Next:** Once this design is approved, start with Phase 1 — Core Plumbing.